

ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

MSc in Big Data Analytics

Distributed Multi-Model Analytics for E-commerce Data

MongoDB + HBase + Apache Spark (PySpark) Analytics System

MSDA9215 Big Data Analytics – Final Project – Technical Report

Lecturer: Temitope Oguntade

Student	Student ID
PATRICK NIYONSHUTI	101420

Submitted in fulfilment of the Final Project requirements

November 2026

Contents

Executive Summary 3

1 Introduction 4
- 1.1 Context and Motivation 4
- 1.2 Objectives 4
- 1.3 Dataset 4

2 System Architecture 5

3 Data Quality Engineering 6
- 3.1 Investigation 6
- 3.2 Critical Finding: Colliding Session Identifiers 6
- 3.3 Cleaning Treatments Applied 6

4 Part 1: Data Modelling and Storage 8
- 4.1 MongoDB Document Model 8
- 4.2 MongoDB Aggregation Pipelines 8
- 4.3 HBase Wide-Column Design 9
- 4.4 HBase Queries 10
- 4.5 Why Data Is Placed in MongoDB or HBase 10

5 Part 2: Data Processing with Apache Spark 11
- 5.1 Batch Pipeline and Optimisation Choices 11
- 5.2 Product Recommendation Indicators 11
- 5.3 Cohort Analysis 12
- 5.4 Spark SQL 13

6 Part 3: Integrated Analytics 14
- 6.1 Customer Lifetime Value 14
- 6.2 Lost Demand: Stock-outs versus Weak Conversion 15

7 Part 4: Visualisation and Business Insights 17
- 7.1 Sales and Funnel 17
- 7.2 Category Performance 18
- 7.3 Key Insights and Recommendations 18

8 Technology Selection: Measured Evidence 19

9 Scalability Considerations 19

10 Limitations and Future Work 20
- 10.1 Limitations 20
- 10.2 Future Work 20

11 Project Requirements: Compliance Summary 21

12 Conclusion 21

Executive Summary

This report documents the design, implementation and findings of a multi-model analytics system for a large e-commerce platform. Each of three technologies is used for what its data model does best. **MongoDB** is the operational document store for products, customers, orders and complete browsing journeys. **HBase** is the wide-column time-series layer for per-customer activity streams and per-product daily performance. **Apache Spark** is the distributed engine for cleaning, large-scale analytics and joining data across the two stores.

The system was built and run end-to-end on the dataset generated by the course script: **10,000** users, **5,000** products, **2,000,000** sessions (**20.0 million** page views) and **500,000** transactions, or 4.0 GB of raw JSON. Every number in this report was produced by that run and is stored in the repository (`output/`, `logs/`).

Three results stand out. First, measured on identical questions, **each engine wins its own access pattern by one to two orders of magnitude**. MongoDB answers a customer's latest sessions in 0.3 ms; Spark answers a 30-day cross-product ranking in 86 ms, against 17.5 s in MongoDB. This is the quantitative justification for the architecture. Second, **stock-outs, not product pages, are the main source of lost revenue**. Half of the catalogue sold out and lost 73% of its exposure, an estimated loss of up to $252.5M against $7.4M from weak conversion. Two earlier methods were tested and rejected along the way, and both are documented. Third, the pipeline **validates its own analytics**. A shuffle test shows that the co-viewing "affinity" is random, and a holdout test shows that customer value cannot be predicted from this data. Both are correct findings about synthetic data; without the tests, the same pipelines would have produced confident-looking recommendations and CLV tiers that are noise.

The report closes with five evidence-based priorities for the business and a discussion of scalability, limitations and future work.

1 Introduction

1.1 Context and Motivation

An online shop generates data of very different shapes. Product catalogues and orders are nested business entities. Click-streams are high-volume, append-only and sparse events. Management questions such as lifetime value or lost demand need all of them at once, at a scale that no single engine serves well. The course framed this as a trade-off: no database is best at everything, and every choice should be justified by the access pattern it serves (Lecture 3A, "NoSQL is an addition, not a replacement").

1.2 Objectives

1. Design and justify MongoDB and HBase schemas for the queries each store will serve (Part 1).
2. Build Spark batch and SQL pipelines for cleaning, recommendation indicators and cohort analysis (Part 2).
3. Answer business questions that need data from several stores at once (Part 3).
4. Communicate findings with visualisations and actionable insights (Part 4).

1.3 Dataset

The data was produced with the course `dataset_generator.py` (seed 42) and spans the 90 days from 13 August to 11 November 2026.

Entity	Records	Key structure
Users	10,000	profile, geography, registration date
Categories	25	3–5 subcategories each, with profit margins
Products	5,000	price history (1–3 points), stock, active flag
Sessions	2,000,000	device, referrer, 4–16 page views, cart, conversion status
Page views (nested)	19,999,622	page type, product, category, view duration
Transactions	500,000	line items, discount, payment method, status

Table 1: Generated dataset (4.0 GB of JSON in 24 files).

One change was made to the generator, and it is documented in the code. The original keeps all two million session dictionaries in a Python list before writing, which needs more than 20 GB of memory. The adapted version writes each 100,000-session chunk as soon as it fills. The output files are identical, and the measured peak memory was **1.04 GB** for the 191-second run.

2 System Architecture

Figure 1 shows how the components fit together. All services ran on one laptop (15 cores, 24 GB RAM): MongoDB 7.0 and HBase 2.1 (with its Thrift gateway) in Docker, and PySpark 4.2 in local mode on 12 cores.

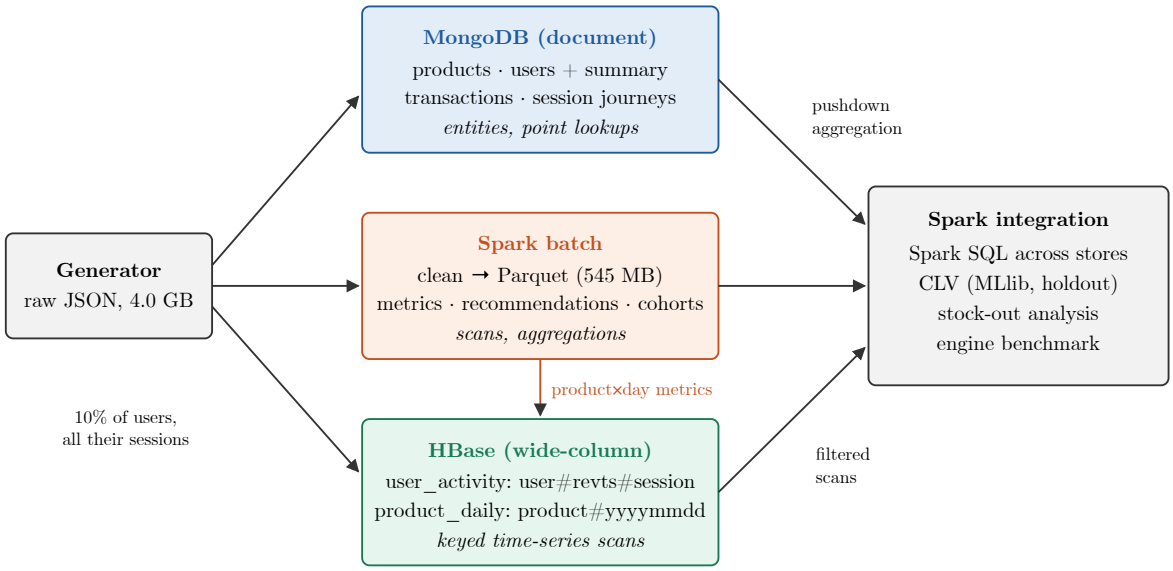


Figure 1: System architecture. Each store serves the access pattern it is designed for; Spark joins them.

The division of labour follows directly from the data models covered in the course:

- **MongoDB** holds entities that are created, updated and read as a whole: an order, a product, a customer profile, one browsing journey. Documents remove joins for these reads, and secondary indexes serve point lookups.
- **HBase** holds append-only event streams that are read by key range and time and are sparse by nature. Its lexicographically sorted row keys turn "latest N" and "between two dates" into one contiguous read, and empty cells cost nothing.
- **Spark** does what neither store does well: scanning and aggregating everything, and joining across sources. It also feeds HBase: the product-day metrics it computes are bulk-loaded into the `product_daily` table.

Data access between the layers uses the native drivers (pymongo, happybase) with work **pushed down** to each store: a MongoDB aggregation, or an HBase scan restricted to one column family and filtered inside the RegionServer. Only reduced results cross into Spark.

3 Data Quality Engineering

3.1 Investigation

Before any analysis, the Spark cleaning job counted every rule violation instead of silently fixing it, and wrote the counts to `output/dq_report.json`. Several of these findings decide what the data can and cannot be used for later in the report.

Check	Result	Consequence
Session-ID collisions	2	IDs are `uuid4().hex[:10]` (40 bits); the birthday bound predicts ≈1.8 collisions at 2M IDs. Both pairs belong to different users.
"Converted" sessions with no transaction	149,276	The conversion *label* overstates conversion: 20.3% labelled vs 12.8% with a recorded order.
Orders not linked to a session	243,389	48.7% of orders and 57.6% of revenue ($268.7M of $466.6M) cannot be attributed to any journey.
Lines sold at a price other than the list price valid at sale time	27.9%	Today's price is stamped on past sales (net effect +$170,914), so price-elasticity analysis is impossible.
Products with `subcategory_id`	0 of 5,000	Shown in the brief but never generated, so margins are estimated at category level.
Non-US country with a US state	9,952	Geography is inconsistent and is used only as a label.
subtotal = Σ items; total = subtotal − discount	0	Monetary fields are internally consistent.

Table 2: Data-quality checks on the generated dataset.

3.2 Critical Finding: Colliding Session Identifiers

The two colliding identifiers (`sess_59df8690f0`, `sess_533c9d2335`) each belong to two different customers in different months. Deduplicating on `session_id` alone, the obvious choice, would therefore have **deleted two genuine sessions**. The cleaning job uses the composite natural key `(session_id, user_id)` instead, and every downstream join follows it. Each store behaved differently. MongoDB's unique `_id` **rejected** both second copies at load time, and the HBase row key, which embeds the user id, stored all four sessions correctly. The defect is negligible in volume but instructive: truncated identifiers are a real-world source of silent data loss.

3.3 Cleaning Treatments Applied

- **Explicit schemas** for every file. Inference would need an extra full pass over 4 GB, and because `cart_contents` is a dictionary keyed by product id, it would have produced a struct with thousands of columns. Declared as `MapType<string,struct>` it is a single column that `explode()` unpacks.
- **Typed timestamps**, trimmed strings with empty strings set to NULL, and upper-cased country and state codes.

- **Normalisation** of nested arrays into tables with `posexplode`: page views, cart items, price history, transaction items and subcategories.
- **Duplicate removal** on natural keys, including the composite session key above.

The clean output is **545 MB of Parquet** (7.3× smaller than the JSON), with `page_views` partitioned by `event_date`. The job runs in 91 seconds.

4 Part 1: Data Modelling and Storage

4.1 MongoDB Document Model

Each collection is shaped around the data that is read together, using the standard document-design patterns.

Collection	Design	Justification
`categories` 25	Subcategories **embedded**	Bounded (3–5) and always read with the parent.
`products` 5,000	Category id and name copied in (**extended reference**); price history embedded	Catalogue pages need the category name without a `$lookup`; price history is bounded (1–3 entries).
`users` 10,000	Profile + **computed** `purchase_summary` + **subset** of the 5 latest orders	Built by an aggregation with `$merge`; segmentation reads one document. Embedding *all* orders would grow without limit (16 MB cap).
`transactions` 500,000	Line items **embedded**, each with a product-name and category snapshot	An order is written and read atomically; the snapshot keeps what the customer actually bought.
`sessions` 2,000,000	Whole journey embedded; cart as an array of objects	One read returns one journey. Product ids used as field names could not be indexed or unwound.

Table 3: MongoDB collections and design rationale.

Indexes follow the queries: `{user_id:1, timestamp:-1}` and a multikey `{items.product_id:1}` on transactions, and `{user_id:1, start_time:-1}` and `{start_time:1}` on sessions. `explain()` confirms an `IXSCAN` with 10 keys and 10 documents examined for "a customer's recent orders" (1 ms). Bulk loading ran at 56,000 documents/s for transactions and about 30,000/s for sessions. WiredTiger compression stored 3.2 GB of session data in 891 MB.

4.2 MongoDB Aggregation Pipelines

P1: revenue by category and month, with each category's top three products. Because line items carry the category snapshot, the pipeline needs no join: `$unwind → $group` (product-month) `→ $sort → $group/$push → $slice`. It runs in 2.2 s over 954,134 line items.

P2: customer segmentation. A first attempt with fixed RFM thresholds produced segments of 47, 2,527, 7,424 and 2 customers, because every synthetic customer orders about 50 times in 90 days. The final pipeline lets the data set the boundaries with `$bucketAuto` (spend quintiles). It reads only the `users` collection, thanks to the embedded summary, and finishes in 17 ms.

Spend quintile	Customers	Avg orders	Avg order value	Revenue
$19,746 – $39,814	2,000	42.1	$855	$71.6M
$39,814 – $44,352	2,000	46.9	$907	$84.4M
$44,352 – $48,483	2,000	50.0	$934	$92.8M
$48,483 – $53,389	2,000	53.0	$963	$101.6M
$53,389 – $80,882	2,000	58.0	$1,007	$116.3M

Table 4: Spend quintiles (P2). The top quintile spends more through both frequency and order value.

P3: most-viewed versus best-selling products (last 30 days). The first version joined to transactions with `let` + `$expr:{$in}`, which *cannot use* the multikey index (30.9 s). Rewriting it with `localField/foreignField` reduced this to 19.7 s. Profiling then showed that 17.8 s of the remainder is spent unwinding 664,457 journeys (6.6M page views). This is the document model's structural weakness: a question *across* many documents rather than *within* one. Section 8 shows the same question answered by Spark in 86 ms.

4.3 HBase Wide-Column Design

Table	Row key	Column families	Reason
`user_activity`	`user_id#(9999999999 − epoch)#session_id`	`s` summary (9 cells); `pv` page views (sparse, qualifier = sequence); `c` cart (sparse, qualifier = product id)	Reversed time sorts a customer's sessions newest-first, so a prefix scan with `LIMIT n` stops early. Each family is a separate store, so a listing never reads the bulky `pv` data.
`product_daily`	`product_id#yyyymmdd`	`m` counters; `dv` views by device; `rf` views by referrer (sparse)	A product's history is one contiguous key range; zero counters are simply not stored.

Table 5: HBase tables. Both are pre-split into five regions, with ROW Bloom filters and `VERSIONS => 1`.

```
def session_row(s):                                    # hbase/load_hbase.py
    rev = 9_999_999_999 - int(start_time.timestamp())  # newest first
    key = f"{s['user_id']}#{rev:010d}#{s['session_id']}"
    cells = {b"s:start": ..., b"s:dur": ..., b"s:status": ...}         # summary
    for i, pv in enumerate(s["page_views"]):                           # sparse
        cells[f"pv:{i:03d}".encode()] = f"{type}|{product}|{category}|{secs}|{offset}"
    for pid, c in s["cart_contents"].items():                          # sparse
        cells[f"c:{pid}".encode()] = f"{c['quantity']}|{c['price']}"
```

Sampling by customer, not by session. `user_activity` holds *all* 200,080 sessions (4.0M cells) of every tenth customer. Sampling 10% of sessions instead would leave every customer's stream 90% incomplete, and the very query this table exists for would be wrong for everyone. The modulo rule spreads the sample over all pre-split regions. `product_daily` holds all 421,769 product-days, loaded from Spark at 25,600 rows/s.

4.4 HBase Queries

The queries implement the scan types from Lecture 4B, in Python (`hbase/queries.py`) and in the HBase shell (`hbase/queries.hbase`). The shell returned identical row counts.

Query	Mechanism	Rows	Time
Last 5 sessions of a customer	prefix scan, family `s`, LIMIT 5	5	3.5 ms (warm)
Sessions in a 14-day window	range scan: start/stop rows from reversed times	26	39 ms
One session's click-stream	`get`, families `pv`, `c`	1	41 ms
Converted sessions only	value scan: `SingleColumnValueFilter`	35	176 ms
Mobile sessions of users 100–199	hybrid: row range + family + value filter	674	152 ms
One product, one month	range scan on `product#yyyymm`	11	31 ms

Table 6: HBase queries on `user_000420` and `prod_00640`.

4.5 Why Data Is Placed in MongoDB or HBase

Entities live in MongoDB and events in HBase. The deciding question is how the data is read. A document is the unit of reading for an order or a journey. A contiguous key range is the unit of reading for "this customer's recent activity" or "this product's last month". In CAP terms both stores are CP as deployed (a single primary or a single RegionServer per region). For a shop this is acceptable: a briefly unavailable recommendation panel is better than a wrong stock level.

5 Part 2: Data Processing with Apache Spark

5.1 Batch Pipeline and Optimisation Choices

Four batch jobs run in sequence: cleaning (Section 3), product-day metrics, recommendation indicators and cohort analysis. The main engineering choices are listed below.

- **Persist only where the lineage branches.** Parsed sessions feed three writes and the quality counts, so they are persisted (`MEMORY_AND_DISK`). Linear pipelines are not cached, following the Lecture 4 rule.
- **`reduceByKey` over `groupByKey`** for pair counting. Counting is commutative and associative, so it can be combined on the map side before the shuffle.
- **Broadcast** the 5,000-entry product table and the per-product counts instead of shuffling them.
- **Single-pass conditional aggregation** for the sparse device and referrer breakdowns, avoiding a discovery job for `pivot()`.
- **Engine settings**: 48 shuffle partitions instead of the default 200, Adaptive Query Execution enabled, 10 GB driver.

One pitfall is worth recording. Passing a plain Python `datetime` into a Spark filter silently converted it from the operating-system time zone (UTC+3), which shifted a 30-day window by three hours and changed a top-20 ranking. Time bounds are now passed as strings and cast inside Spark; the corrected results match MongoDB exactly.

5.2 Product Recommendation Indicators

```
pairs = (baskets_rdd                          # products viewed per session
         .flatMap(lambda items: itertools.combinations(items, 2))
         .map(lambda pair: (pair, 1))
         .reduceByKey(lambda a, b: a + b))   # combine before shuffle

lift(A,B) = n(A,B) · N / (n(A) · n(B))       # > 1 = affinity beyond chance
```

Raw co-occurrence counts favour popular products, so each pair is scored by **lift**. To check whether the lifts mean anything, the job adds a **shuffle test**: product ids are randomly permuted across all baskets, keeping basket sizes and product popularity, and the pairs are recounted. If observed lifts look like the shuffled ones, there is no real affinity to recommend.

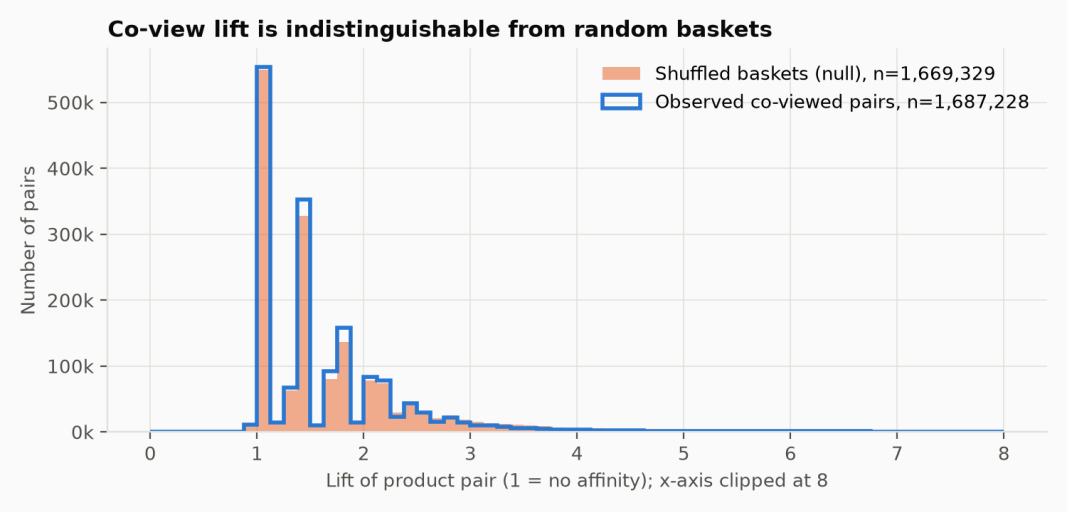


Figure 2: Lift of 1,687,228 observed co-viewed pairs against 1,669,329 pairs from shuffled baskets. Medians are 1.428 and 1.431; the 95th percentiles are 3.26 and 3.61.

The distributions are indistinguishable (Figure 2). The highest observed lifts, up to 560, belong to pairs seen only three times. Co-purchase tells the same story: no pair appears together in more than five of 500,000 orders. The generator draws products uniformly at random, so this is the correct finding. Recommending these pairs would be recommending noise. The value of the job is the test itself, which will show whether real data contains signal.

5.3 Cohort Analysis

Customers are grouped by registration month (February–August 2026) and followed through each activity month. Raw monthly revenue per member swung by about ±65%, but this was an artefact of the calendar: the window starts on 13 August and ends on 11 November, so the first and last months are partial. After normalising by the number of days observed, every cohort places **0.55 orders per member per day**, within ±6%, at every age, with retention of essentially 100% (Figure 3). There is no cohort effect: in this data a customer's behaviour does not depend on when they joined.

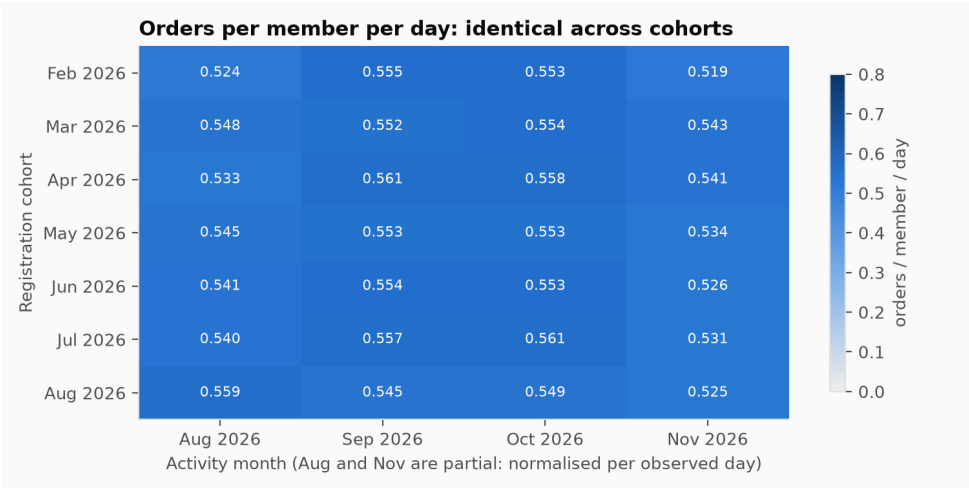


Figure 3: Orders per member per day by registration cohort and activity month. The colour scale is anchored at zero, so equal colours are the finding.

5.4 Spark SQL

The cleaned tables are registered as views and queried with SQL (`spark/05_spark_sql.py`):

- a **device × referrer funnel** (CTE with conditional aggregation). Conversion is 20.0–20.5% in all 15 combinations, so channel makes no difference;
- the **best product per category** with `RANK() OVER (PARTITION BY category)` and its share of category revenue;
- a **price-at-time-of-sale audit**: a non-equi join to `price_history` with `ROW_NUMBER()` picks the price valid at each order's timestamp, which is the source of the 27.9% finding in Table 2;
- **inter-purchase intervals** with `LAG()` (median 1.22 days, interquartile range 0.51–2.44);
- **weekly revenue and estimated gross profit** by category.

SQL across the operational stores. MongoDB customers with their embedded summary (10,000 documents in 0.15 s) and the HBase `product_daily` series (421,769 rows in 4.6 s) are loaded into DataFrames, registered as views and joined in a single statement with the Parquet line items. The query ranks countries by customer value and attaches each country's favourite product with its lifetime funnel from HBase. The MongoDB and HBase Spark connectors would make these reads parallel at production scale. At these result sizes a driver-side read costs nothing measurable, and it avoids version-pinned connector jars.

6 Part 3: Integrated Analytics

6.1 Customer Lifetime Value

Business question. Which customers will be worth most next quarter, and does on-site engagement predict future spend *beyond* purchase history? If it does not, the nightly join of 2M sessions to the customer table can be dropped from the CLV pipeline.

Data sources and workflow.

1. **MongoDB** `transactions`: one pushed-down aggregation splits each customer's spend at a cutoff (0.46 s).
2. **HBase** `user_activity`: family `s` only, with a server-side `SingleColumnValueFilter` on `s:start < cutoff` (133,770 rows in 2.6 s; ISO timestamps compare correctly as bytes).
3. **Parquet** line items: a category-weighted estimated margin for each customer.
4. **Spark**: join, then train two MLlib linear regressions on a 70/30 split of the 1,000 HBase-sampled customers.

The 90 days are split into a 60-day **calibration** period (features) and a 30-day **holdout** (target). Model M1 uses purchase history (frequency, monetary value, recency, tenure); M2 adds the four HBase engagement features. CLV(12 months) = predicted 30-day spend × 12 × estimated margin.

Model	Test R^2	Test MAE	Test RMSE
Naive rule: next 30 days = ½ × previous 60 days	–	$4,357	–
M1: purchase history	–0.010	$3,523	$4,305
M2: history + HBase engagement	–0.018	$3,527	$4,323

Table 7: Holdout performance of the CLV models.

Result. Neither model explains future spend. R^2 is about zero, and every feature's correlation with holdout spend is below |0.04|. The models beat the naive rule only by predicting the average. The top predicted decile delivered 10.0% of actual holdout spend, which is exactly what chance gives (Figure 4). This is a property of the data, not a modelling failure: the generator assigns every session and order to `random.choice(users)`, so past behaviour carries no information about the future. Without the holdout, the same pipeline would have produced convincing-looking CLV tiers that are noise.

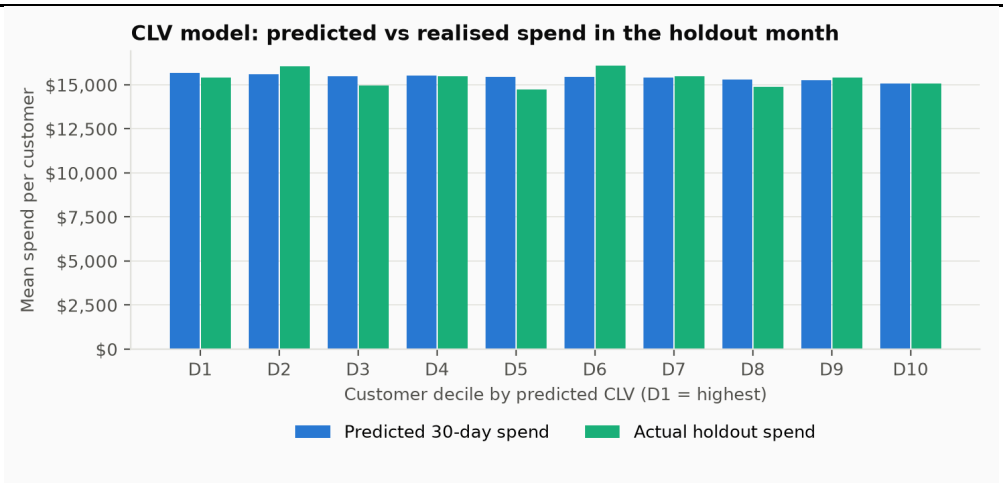


Figure 4: Predicted 30-day spend against realised holdout spend by predicted-CLV decile. The flat pattern shows the model cannot rank customers.

6.2 Lost Demand: Stock-outs versus Weak Conversion

Business question. How much revenue was lost because products ran out of stock, and which in-stock products convert below their peers? **Data sources.** HBase `product_daily` (views, cart additions and units per product per day), MongoDB `products` (stock, active flag, price, category), and Spark for joins and per-category medians. Three methods were tried. The first two were rejected, and the reasons are part of the result:

1. **Units-per-view against the category median, all products: $125.9M "leaked". Rejected.** Half the catalogue (2,492 products) sold out, and the site almost stops showing a product once its stock is zero. Sold-out products keep the sales they made but lose their views, so their units-per-view is inflated (0.46 vs 0.22). That raises every category median and makes in-stock products look leaky. The confounder is inventory.
2. **Event study around each product's sell-out day. Rejected.** Views of sold-out products are spread evenly over the 90 days (second-half/first-half ratio 1.02, identical to in-stock products), and the computed sell-out days cluster at the end. The generator depletes stock in *generation* order but stamps each session with a *random* date, so the inventory history contradicts the data's own clock, and no time-series method can work.
3. **Cross-sectional comparison within category (adopted).** Missing views = median views of in-stock peers − own views. Lost units = missing views × the peers' median units-per-view. Lost revenue = lost units × price. Conversion leakage is measured among in-stock products only, against in-stock medians.

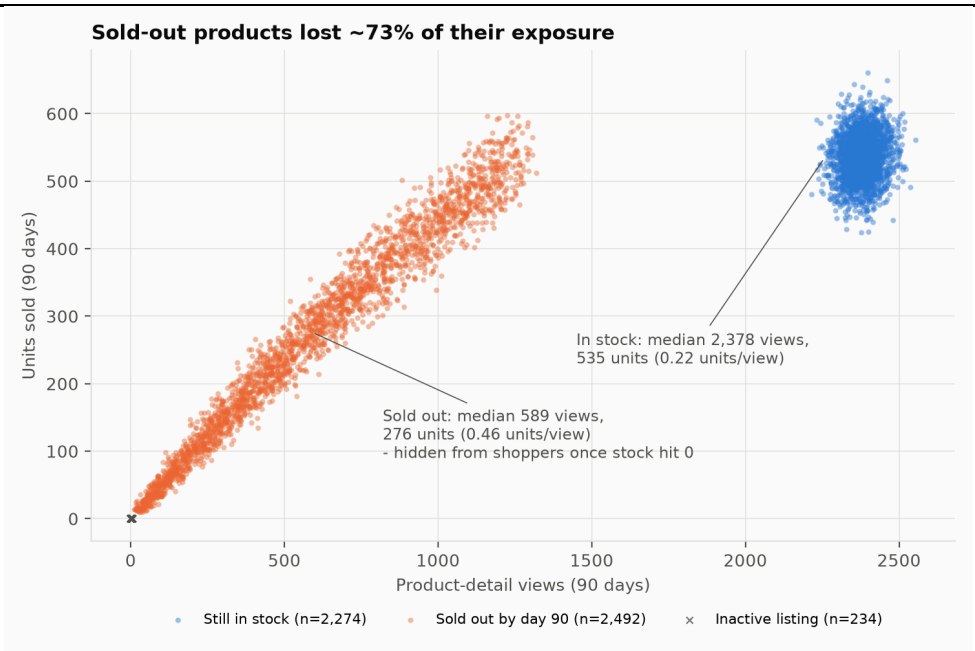


Figure 5: Views against units sold per product. In-stock products form one cluster at about 2,400 views; sold-out products lost about 73% of their exposure once they disappeared from the site.

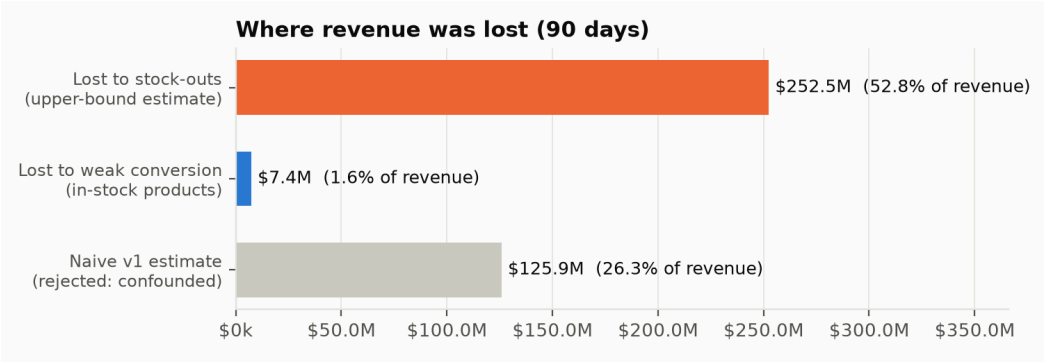


Figure 6: Estimated revenue lost in 90 days. Stock-outs cost about 34 times more than weak conversion. The rejected naive estimate is shown for contrast.

Result. The estimated loss from stock-outs is **$252.5M**. This is an upper bound, equal to 52.8% of realised revenue, and assumes a sold-out product would have attracted and converted traffic like its in-stock peers. Weak conversion among in-stock products costs only **$7.4M** (1.6%). Every category has 41–58% of its products sold out, so the problem is systemic rather than category-specific: initial stock is drawn uniformly from 10–1,000 units against about 500 units of demand per product. Inactive listings received only 322 views in total and do not matter.

7 Part 4: Visualisation and Business Insights

7.1 Sales and Funnel

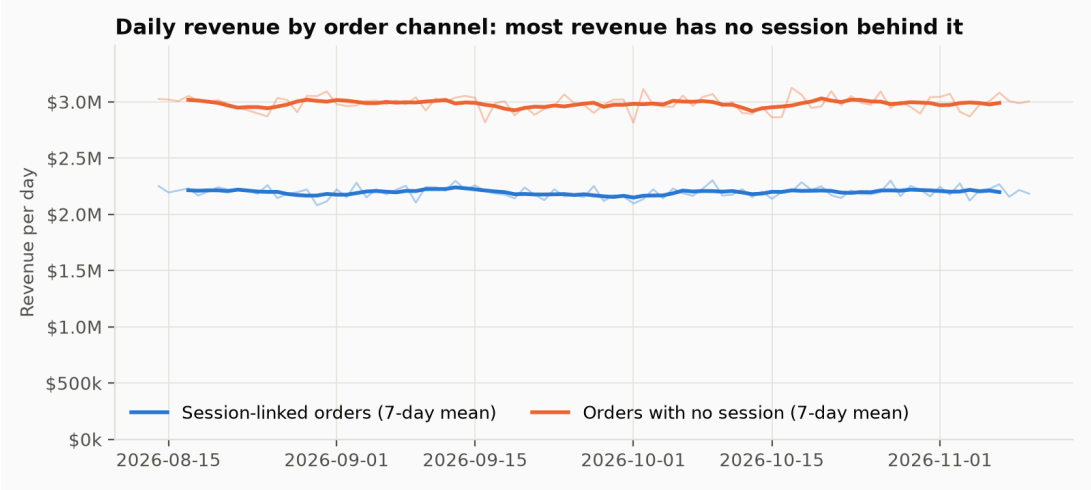


Figure 7: Daily revenue by order channel. Orders with no session behind them bring in more revenue every day ($3.0M against $2.2M), so journey analytics can explain less than half of sales.

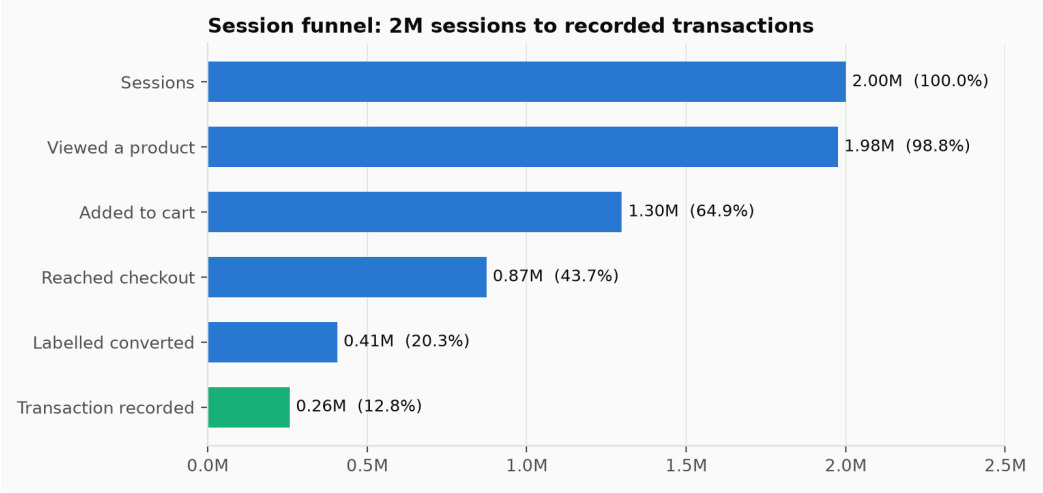


Figure 8: Session funnel. The gap between "labelled converted" (20.3%) and "transaction recorded" (12.8%) is the conversion-label defect from Table 2.

7.2 Category Performance

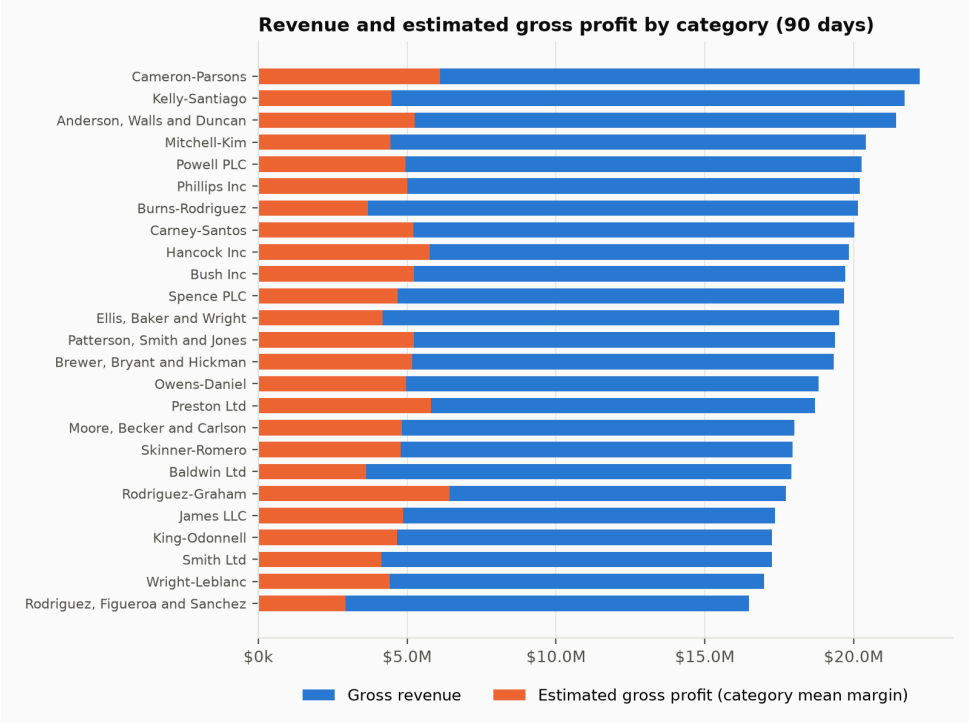


Figure 9: Revenue and estimated gross profit by category. Revenue is spread evenly ($16.5–22.2M), but estimated margins range from 17.8% to 36.3%, so the profit ranking differs from the revenue ranking (Spearman 0.34). Kelly-Santiago is second by revenue but eighteenth by profit.

7.3 Key Insights and Recommendations

1. **Fix inventory first.** Half the catalogue sold out and lost 73% of its exposure; stock-outs are by far the largest revenue lever (up to $252M against $7M). Reorder points can be set from the daily sell-through already held in HBase `product_daily`.
2. **Manage categories on profit, not revenue.** With margins of 17.8–36.3%, the revenue leaderboard misleads.
3. **Link the 58% of revenue that has no session behind it.** Attribution and journey analytics cannot see most of the business until those orders are traceable.
4. **Treat conversion labels and historical prices as unreliable** until the upstream pipeline is fixed (149,276 false conversions; 27.9% of lines at the wrong price).
5. **Do not deploy recommendations or CLV tiers on this data.** The shuffle test and the holdout both show no signal. Channel, device and cohort show no effect either, so marketing spend cannot be targeted on them. The same pipelines, with the same validation steps, apply unchanged to real data.

8 Technology Selection: Measured Evidence

To turn the design arguments into evidence, the same two questions were run on all three engines (`integration/benchmark_engines.py`; best of three warm runs; answers verified identical across engines).

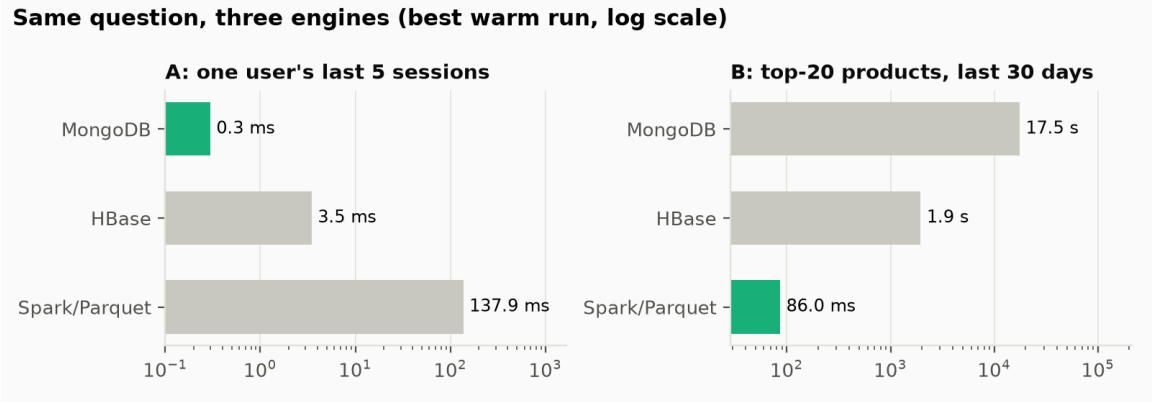


Figure 10: Query time by engine, on a log scale. HBase runs under x86 emulation on this machine, so its times are pessimistic.

Question	MongoDB	HBase	Spark/Parquet
A: one customer's last five sessions	**0.3 ms**	3.5 ms	137.9 ms
B: top 20 products, last 30 days	17,494 ms	1,943 ms	**86 ms**

Table 8: Benchmark results.

For a point query, MongoDB's B-tree index is fastest and HBase's row-key prefix scan is close behind; Spark has to scan a table and is the wrong tool. For an analytical question across all products, Spark with Parquet partition pruning wins by two orders of magnitude. MongoDB must unwind 664,000 documents, and HBase must scan `product_daily` in full because its row key is product-first. A date-first key would answer question B quickly but would concentrate every write on "today": a row key can serve one dominant access pattern well, and this one serves per-product history. The measurements confirm the architecture: **MongoDB for entities, HBase for keyed time series, Spark for scans and joins.**

9 Scalability Considerations

- **Spark.** All jobs are expressed as distributed transformations except two deliberately small driver-side steps: the 5,000-entry broadcast, and the shuffle-test permutation, which at 100× scale should become a `rand()`-keyed distributed shuffle. On a cluster the JSON-to-Parquet step would read from HDFS or S3 with partitions of about 128 MB.
- **MongoDB.** Shard `transactions` and `sessions` on a **hashed** `user_id`. This gives an even spread while keeping per-customer queries on one shard; a ranged time key would hotspot inserts. Cross-journey aggregations such as P3 do not scale on any shard key and belong in Spark.

- **HBase.** Regions split automatically. Sequential customer ids would hotspot during loading, which pre-splitting mitigated; at 50× scale a salt prefix (`hash(user) mod N`) would be added. Enabling Snappy or ZSTD compression, which this image lacks, would reduce the 184 MB per 200,000 sessions by roughly 3–4×.
- **Integration.** Replace driver-side reads with the MongoDB Spark Connector and HBase-Spark bulk scans for parallel, locality-aware reads.

10 Limitations and Future Work

10.1 Limitations

- **Synthetic data.** Customers, products and timestamps are drawn independently. Several analyses therefore return well-supported *negative* results that demonstrate method rather than real customer behaviour.
- **Single-node deployment.** Spark in local mode, one MongoDB instance and standalone HBase under emulation. Absolute timings are indicative; the order-of-magnitude differences are the result.
- The stock-out loss is an **upper bound** under stated assumptions. Margins are category-level estimates because products lack subcategories, and the CLV formula ignores discounting and churn.
- HBase holds a 10% customer sample of sessions, so the CLV model is trained on 1,000 customers.

10.2 Future Work

- Structured Streaming from a message broker (MQTT or Kafka, as in the course IoT pipeline) into HBase counters, for real-time product metrics and stock alerts.
- MongoDB change streams to keep `purchase_summary` current without a batch rebuild.
- Probabilistic CLV (BG/NBD with Gamma-Gamma) and collaborative filtering (ALS) on real data, each deployed only after its validation test shows signal.

11 Project Requirements: Compliance Summary

Requirement (project brief)	Where it is met
MongoDB model: catalogue, users, transactions	§4.1; `mongodb/load_mongo.py`
Two non-trivial aggregation pipelines	§4.2 P1–P3; `mongodb/aggregations.py`
HBase column families for sessions and product metrics	§4.3; `hbase/schema.hbase`
Load sessions; retrieve one user's activity	§4.3–4.4; `hbase/load_hbase.py`, `queries.py`, `queries.hbase`
Spark cleaning and normalisation	§3; `spark/01_clean_normalize.py`
Recommendation indicators or cohort analysis	Both: §5.2–5.3; `spark/03_*.py`, `04_*.py`
Spark SQL, including data from MongoDB and HBase	§5.4; `spark/05_spark_sql.py`, `stores.py`
Integrated analytical query	Two: §6.1 CLV, §6.2 lost demand; `integration/`
3–4 visualisations	Ten figures; `viz/make_figures.py`
Scalability and technology justification	§8–9, with measurements
GitHub repository with reproduction instructions	`README.md`, `run_all.sh`

Table 9: Mapping of the project brief to this report and the repository.

12 Conclusion

The project shows that a multi-model architecture is a set of deliberate trade-offs, each of which can be measured. MongoDB serves whole entities in sub-millisecond time, HBase serves keyed time series through row-key design, and Spark serves anything that has to look at everything. Together they answered the brief's business questions over two million sessions on a single laptop. The most useful result, though, is methodological. The first version of three analyses gave a confident and wrong answer: fixed segment thresholds, raw cohort months and the naive leakage estimate. Each was caught by questioning the data rather than the code. The resulting picture is honest: inventory is the binding constraint, profit and revenue rank categories differently, and in this dataset there is no customer-level signal to monetise yet. The pipelines and their validation steps are ready for data that has one.

Reproducibility. `./run_all.sh` rebuilds every result in about 15 minutes. Repository: github.com/Patrick101420/auca-msda9215-ecommerce-analytics.

Use of AI assistance. An AI assistant (Claude, Anthropic) was used to help write and debug the code and to draft this report. All code was executed on the author's machine, and every result was produced by those runs and checked against the output files.